

```

System.out.println("c=repository, s=find, r="+name);
return name;
}

```

With these changes, the *InMemoryUserRepository* is equipped with a journaling feature.

Plug-and-play

An interesting observation is that the journaling feature is not a core feature of UMS. Only some customers may show interest in this feature while others do not. In other words, we cannot permanently change the code of *InMemoryUserRepository* with or without the journaling feature. Instead, journaling must be built only as a pluggable feature to the *InMemoryUserRepository*.

Usually, a pluggable feature is expected to be added or removed from an existing system just through configuration, without making any code changes. There are many such features like security, validation, transaction management, session management, encoding/decoding, encryption/decryption, etc, which are good candidates for plug-and-play.

In fact, any optional feature that can be implemented by the way of pre-processing and post-processing can be a plug-and-play feature. It is one of the basic essences of a modular system. The *proxy* pattern explains the design for such a pluggable module.

Proxy

As the name suggests, a proxy acts as a frontend to someone else, which we call a 'target'. However, not every frontend to a target can be a proxy. There are a couple of expectations with respect to a proxy:

1. A proxy must look the same as a target, for the outsiders. In fact, for them, a proxy is not even noticeable. In other words, the interface of the proxy must be the same as that of the target.
2. A proxy must not replace the target. Instead, it only offloads some optional work from the target and leaves the core functionality with it. In other words, a proxy only takes care of pre-processing and post-processing, leaving the business logic to the target.

Going by these observations, let us redesign the UMS.

We will introduce a class named *UserRepositoryJournalProxy* to act as a proxy to any implementation of the *UserRepository* interface, not just only for *InMemoryUserRepository*. So, the *UserRepositoryJournalProxy* implements the *UserRepository* interface.

Apart from that, the *UserRepositoryJournalProxy* also holds a reference to *UserRepository*, which acts as the target.

Consumers like the *Application* in Figure 1 still refer to the *UserRepository* interface like in the past. Remember, there

should not be any impact on the consumer with or without the presence of a proxy.

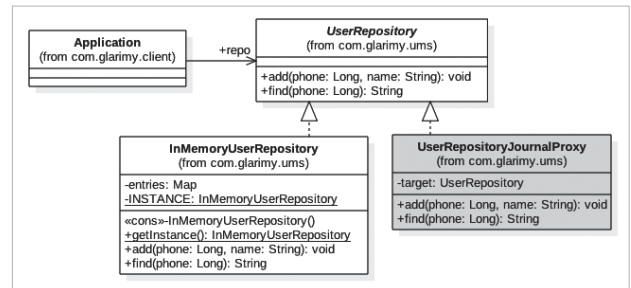


Figure 1: Proxy pattern

Since we plan to offload the journaling work to the *UserRepositoryJournalProxy*, the *InMemoryUserRepository* limits itself only to the core business functionality which is in no way different from what we have done in Part 1.

```

public void add(Long phone, String name) {
    entries.put(phone, name);
}

public String find(Long phone) {
    return entries.get(phone);
}

```

A proxy always requires a target of the same interface like the *InMemoryUserRepository*. The target can be injected into the *UserRepositoryJournalProxy* through the constructor.

```

public class UserRepositoryJournalProxy implements
UserRepository {
    private UserRepository target;

    public UserRepositoryJournalProxy(UserRepository target) {
        this.target = target;
    }
    ...
}

```

Notice that only the proxy is aware of its target, while the target is unaware of the presence of the proxy.

Now is the final step. Like any other proxy, the *UserRepositoryJournalProxy* should delegate the core functionality to its target and handle only the pre-processing and post-processing, which in this case is to log audit records to the console.

```

public void add(Long phone, String name) {
    System.out.println("c=repository, s=add, a=" + phone + "&"
+ name);
}

```